

Deconstructing the Avoidance of Uninitialized Reads of Auto Variables

Document #: P2754R0
Date: 2023-01-24
Project: Programming Language C++
Audience: isocpp-ext
Reply-to: Jake Fevold
[<jfevold@bloomberg.net>](mailto:jfevold@bloomberg.net)

Contents

- [1 Introduction](#)
- [2 Open Questions of Scope](#)
- [3 Properties of a Solution](#)
- [4 Relevant Code Examples](#)
 - [4.1 Reading an Uninitialized Automatic Variable Directly](#)
 - [4.2 Reading an Uninitialized Automatic Variable Conditionally](#)
 - [4.3 Passing an Uninitialized Automatic Variable to Another Function by Value](#)
 - [4.4 Passing a Potentially Uninitialized Automatic Variable to Another Function by Value](#)
 - [4.5 Passing an Uninitialized Automatic Variable to Another Function by Reference or Pointer](#)
 - [4.6 Common Idioms Delaying Initialization for Performance](#)
 - [4.7 Initialization for Template Types Leaves Room for Improvement](#)
- [5 Suggested Solutions](#)
 - [5.1 Always Zero-Initialize](#)
 - [5.2 Zero-Initialize or Diagnose](#)
 - [5.3 Force Initialization in Source](#)
 - [5.4 Force Initialization or Explicit Delayed Initialization Intent in Source](#)
 - [5.5 Initialize to Implementation-Defined Value, But Read Before Write Is Still Undefined Behavior](#)

- 5.6 Initialize to Implementation-Defined Value, But Read Before Write Is Erroneous Behavior
- 5.7 Value-Initialize Only
- 6 Conclusion

§ Abstract

Inadvertently reading uninitialized values of automatic variables on the program stack presents a common source of security holes. An initial suggestion was proposed in [P2723R0] as “an opening bid” to get the discussion started. As a result, a variety of thoughtful proposals have been offered regarding how the C++ language might be altered to minimize or mitigate such occurrences. These suggestions have varying tradeoffs, especially with respect to safety versus correctness and the extent to which initial intent is preserved. We attempt to survey the various alternatives that have been proposed on the The WG21 Reflector and objectively contrast their respective properties in the hope of facilitating a productive and fruitful exchange of ideas and opinions at the February 2023 Standards meeting and in the future.

§ 1 Introduction

This paper is meant to drive fruitful discussion of the topics described in JF Bastien’s paper, “Zero-initialize objects of automatic storage duration” [P2723R0]. The stated goal of Bastien’s paper is to eliminate, where possible, security problems stemming from an *indeterminate value* held by variables of nonclass type of automatic storage duration. *Nonclass* types are *arithmetic* type, *pointer-to-object* type, *pointer-to-function* type, *pointer-to-data-member* type, *pointer-to-member-function* type, *enumeration* type, `std::nullptr_t` (since C++11), and *POD* (before C++11), as well as *array* (of any of these types).

A similar problem exists with uninitialized dynamic memory (such as that returned from `malloc`, `operator new`, and similar allocation utilities), which is not addressed in Bastien’s paper nor here.

§ 2 Open Questions of Scope

Considering the various suggested mitigation strategies for security holes due to typical nonclass variables raised several other questions regarding scope. That is, should a proposed solution to address scalar and pointer types also apply to:

- Dynamic memory returned from `malloc`, `operator new`, and similar?
- Arrays?
- Bytes of a union that are not used by the currently selected type?

- Padding in the object representation?

These considerations of scope seem mostly orthogonal to choosing a basic direction and are not discussed further here.

§ 3 Properties of a Solution

As part of curating the information provided by proposals and discussions in The WG21 Reflector, several recurring questions were used to nail down the salient properties and features of a given proposal. A well-considered, though non-exhaustive, curation of such questions is presented here.

- Does old code that has undefined behavior because of reads of uninitialized values but is known or believed to work acceptably continue to compile? If so, under what circumstances (e.g., compiler flags and so on)?
- Does the solution add behaviors that are not well defined today? For example, does `int i;` go from meaning “uninitialized” to meaning “initialized to zero”?
- Does the solution *change* the semantics (i.e., observable behavior) of any existing program?
 - A program that is correct today?
 - A program that appears to be correct today but isn’t?
- Does quality of implementation play a role in what programs are ill formed? If one compiler is capable of proving that a value is read before being initialized, can it fail to compile whereas another compiler initializes the value and proceeds?
- Do tools that are not compiler integrated, like Valgrind, continue to be usable to detect (i.e., as program defects) semantic reads of uninitialized variables?
- Is reading from a variable without first explicitly initializing or writing to that variable still incorrect? If not, what default values are appropriate for pointer-to-member and similar types?
- Is an opt-out annotation, per variable, available in the source?
- Is an opt-out tooling annotation (i.e., compiler flag) included?
- Does a given solution allow for evolution to a better solution, such as a simpler initialization model?

§ 4 Relevant Code Examples

The examples in this section serve to elucidate various common security or correctness defects that a viable proposal might address.

§ 4.1 Reading an Uninitialized Automatic Variable Directly

```
void f1()
{
    int p;
    int q = p + 1;  // UB
}
```

The C++ code snippet above is clearly and unconditionally incorrect today.

§ 4.2 Reading an Uninitialized Automatic Variable Conditionally

```
void f2()
{
    int y;
    int z = b ? y + 1 : 0;
}
```

If `b` is never true, then this code is technically correct today. If `b` is never true, then the conditional serves no purpose. `int z = 0;` is unconditionally an improvement.

§ 4.3 Passing an Uninitialized Automatic Variable to Another Function by Value

```
void g3(int);
void f3()
{
    int x;
    g3(x);  // likely a bug
}
```

The C++ code snippet above is likely a bug. If the author is certain `g3` doesn't use the value of the argument, then a literal would suffice.

§ 4.4 Passing a Potentially Uninitialized Automatic Variable to Another Function by Value

```
void g4(int);
void f4()
{
    int s;
    if (c) s = 0;
    g4(s);  // likely a bug
}
```

If `g4` uses the value of the parameter only when `c` is true, then this example code is correct today. Because `g4` doesn't take the value of `c` as a parameter, the example code likely has a bug.

§ 4.5 Passing an Uninitialized Automatic Variable to Another Function by Reference or Pointer

```
void g5(int*);
void f5()
{
    int t;
    g5(&t); // possibly a bug
}
```

If `t` is strictly an output of `g5`, this example code is correct today. If `g5` compares the address of `t` but does not dereference the pointer, this example code is correct today. Compilers cannot currently reason about the contract of `g5` when the definition of `g5` is in a different translation unit.

§ 4.6 Common Idioms Delaying Initialization for Performance

```
void f6()
{
    char buffer[1000];
    BufferAllocator a(buffer, sizeof buffer);
    std::vector v(&a);

    char buffer2[1000];
    snprintf(buffer2, sizeof buffer2, "cstring");
}
```

Idioms like these are safe and efficient and are not a common source of security concerns.

§ 4.7 Initialization for Template Types Leaves Room for Improvement

```
template <typename T>
void f7()
{
    T t;
    cout << t;
}
```

For class types, `t` is initialized and the C++ code snippet above is correct today. For primitive types, `t` is uninitialized and the behavior of `f7` is undefined.

§ 5 Suggested Solutions

Each solution is evaluated for *viability*, *backward compatibility*, and *expressability*.

Viability is an evaluation of whether a solution is logically consistent, both internally and with respect to the existing C++ Standard. Viability is summarized for each solution as either *viable*, *nonviable*, or *unclear*. *Viable* means that a given solution is consistent. *Nonviable* means that a given solution is inconsistent either with itself or with other

foundational rules or definitions in the Standard. *Unclear* means that the available information is insufficient for making a sound determination.

Backward compatibility is an evaluation of whether all existing, compiling code would continue to compile and behave as it does now if a given solution were adopted. Note, if buggy code continues to compile and behave identically, then the root security problem is unaddressed. Backward compatibility is summarized as either *compatible*, *correct-code compatible*, *incompatible*, or *unclear*. *Compatible* means that, if a given solution were adopted, all code which previously compiled continues to compile, with behavior differences only in the case of previously undefined behavior (UB). *Correct-code compatible* means that all previous correct code compiles, but some or all code that had UB would not compile. *Incompatible* means that some previously correct code would not compile. *Unclear* means that the available information is insufficient for making a sound determination.

Expressability is an evaluation of whether previously existing code would maintain its current meaning if a given solution were adopted. Currently, an uninitialized automatic nonclass variable declaration could be either an inadvertent, logical error (e.g., the original author meant to initialize but didn't), or an intentional, delayed initialization. Expressability is summarized as either *better*, *unchanged*, *worse*, or *unclear*. *Better* means that previously existing code must be updated to make explicit the intent to delay initialization or correct the logical error. *Unchanged* means that previously existing code would be no more or less ambiguous. *Worse* means that previously existing code would be more ambiguous because logical error and intentionally delayed initialization are no longer the only two possibilities. *Unclear* means that the available information is insufficient for making a sound determination.

Note that some solutions list additional concerns that are not generally applicable to other solutions.

§ 5.1 Always Zero-Initialize

All uninitialized automatic-storage-duration nonclass variables are initialized to a specific value. Numerical types would be initialized to zero. The value for pointer types is an open question. Major compilers offer an option to zero-initialize already.

- **Viability:** *Viable*. This solution has already been implemented and is viable, concrete, and easily understood.
- **Backward Compatibility:** *Compatible*. All existing code continues to compile, and all existing correct code continues to work correctly. Behavior of some existing code that was previously undefined becomes defined, though that now-defined behavior might not be correct. Importantly, incorrect code that was previously working properly might now exhibit different, unexpected behavior, which could be better or worse.

- **Expressability:** *Worse.* Currently, a declaration without an initialization is either an accidental omission or an intentional delayed initialization, meaning a promise to write before read. Going forward, a declaration without an initialization will be indistinguishable between an unintentional failure to initialize and an intentional zero initialization. All the examples listed become well defined in all branches. For existing bugs, the new well-defined behavior might, by happenstance, be the intended behavior, or it might not.
- *Other Concerns:* Tooling. No tools will be able to detect existing logical errors since they will become indistinguishable from intentional zero initialization. The declarations `int i;` and `int i = 0;` would have precisely the same meaning.

§ 5.2 Zero-Initialize or Diagnose

Code having an unconditional read of an indeterminate value is diagnosed (i.e., rejected), and code with a potential read of an indeterminate value must zero-initialize the variable and accept the code as well formed.

- **Viability:** *Unclear.* Whether this solution is viable depends on the verbiage with respect to the abstract machine, for which no proposal is currently available. Rejecting code in which indeterminate value is unconditionally read relies on the quality of the implementation. Stating both that the value of an uninitialized variable is zero *and* that the behavior of reading that value is undefined is inconsistent. Also problematic is stating that the result of reading an uninitialized variable is, depending on the implementation, either (1) well defined (as having a value of zero) or (2) disallowed.
- **Backward Compatibility:** *Correct-Code Compatible.* Some number of existing bugs that would previously compile will now fail to compile, specifically some but not necessarily all bugs where a read of indeterminate value is unconditional. All bugs conditional on runtime values will continue to compile, and will have deterministic outcomes, which might not be correct and might even cause a program that appears to be working to suddenly exhibit different, unexpected behavior.
- **Expressability:** *Unchanged.* By allowing a diagnostic, the semantics of uninitialized variables remain unchanged.
- *Other Concerns:* Validity becomes dependent on quality of implementation. This solution would introduce a condition where code that is accepted by one conformant compiler might not be accepted by another.

§ 5.3 Force Initialization in Source

All uninitialized automatic-storage-duration nonclass variables are ill formed. Delayed initialization would require use of `std::optional` or a similar mechanism.

- **Viability:** *Viable.* The solution is viable, concrete, and easily understood.

- **Backward Compatibility:** *Incompatible*. Any existing code having delayed initialization of automatic-storage-duration nonclass variables would need to be updated.
- **Expressability:** *Better*. Accidental omission of an initial value is no longer possible; one must explicitly choose a class type or provide an initial value. Consequently, a person must evaluate existing code and make changes. Note that using a script to initialize every uninitialized automatic variable would be just another means of masking the original author's intent.

§ 5.4 Force Initialization or Explicit Delayed Initialization Intent in Source

All uninitialized automatic-storage-duration nonclass variables are ill formed unless specifically annotated. A suitable syntax must be chosen to specify when leaving a variable uninitialized is deemed necessary.

- **Viability:** *Viable*. The solution is viable, concrete, and easily understood.
- **Backward Compatibility:** *Incompatible*. Any existing code with delayed initialization of automatic-storage-duration nonclass variables would again need to be updated. A tool to annotate all such variables as uninitialized could, however, be easily employed for use cases in which security is not deemed important.
- **Expressability:** *Better*. Accidental omission of an initial value is again no longer possible; one must explicitly choose to annotate or give an initial value. Improvement of existing code is dependent on the quality of the updates. If all previously uninitialized variables are mindlessly annotated as intentionally delayed, then, in practice, correctness bugs become harder to find.

§ 5.5 Initialize to Implementation-Defined Value, But Read Before Write Is Still Undefined Behavior

All uninitialized automatic-storage-duration nonclass variables are initialized to an implementation-defined value, but reading that value is still UB. One could, in development and testing, inject values that are likely to cause noticeable failures (e.g., signaling NaN, unaligned pointer, and so on) and, in production, inject best-guess values, such as zero for integers.

- **Viability:** *Nonviable*. Undefined behavior has a specific meaning; declaring behavior undefined and also defining some aspects of the behavior is inconsistent with that meaning. Although we might recommend that all vendors follow this guidance, a compiler that failed to do so (e.g., to optimize performance) would nonetheless remain conforming.
- **Backward Compatibility:** *Compatible*. All existing code still compiles, and all existing correct code works as it did before. Again, compilers that follow this

suggestion might well expose defects in programs that previously were behaving as expected for all inputs.

- **Expressability:** *Unchanged*. Everything means exactly what it did before; only the results of previously UB are allowed (and encouraged) to change.

§ 5.6 Initialize to Implementation-Defined Value, But Read Before Write Is *Erroneous Behavior*

All uninitialized automatic-storage-duration nonclass variables are initialized to an implementation-defined value, yet reading that value is always wrong (like *UB*) but still defined (unlike *UB*). The original term proposed for this defined but undesirable behavior is *erroneous behavior* (*EB*). Any program that contains *EB* is incorrect, but the behavior is *implementation defined*. Different projects could elect to treat *EB* as *UB* for performance, could use hostile default values for testing and development, or could use somewhat safe default values for production.

- **Viability:** *Viable*. The solution is viable because *EB* is separate from *UB*, thus avoiding inconsistency. Getting the wording exactly right might be challenging.
- **Backward Compatibility:** *Compatible*. Existing, correct programs continue to work exactly as before. Existing programs with bugs continue to compile, all existing options for finding bugs continue to be viable, and opportunities for new tools become available as well. Again, defects in apparently working programs might manifest as the result of this change, and programs that had observable defects might suddenly start behaving as intended.
- **Expressability:** *Unchanged*. Everything means exactly what it did before.

§ 5.7 Value-Initialize Only

Remove default initialization entirely and have only value initialization. The state of initialization in C++ is already very complex, and the cost of this complexity is dubious for the level of utility it affords. The entire initialization system could be pared down to one single form of initialization that provides values in all cases. This more fundamental change addresses uninitialized-variable problems, as well as other known issues with initialization.

- **Viability:** *Unclear*. While this bold general reimagining of C++ initialization might be the ideal solution, many committee members would agree that getting it right would take far more time than any other solution presented here. Some consider the security concerns being addressed as distinctly urgent, so waiting for a broader-scope solution might be considered unacceptable. Arguably, no solution to the narrow-scope problem should be undertaken if that narrow-scope solution would substantially restrict future options for a much improved, wider-scope solution to the complexity-of-initialization problem.

- **Backward Compatibility:** *Unclear*. Without more information on the specifics, backward compatibility is difficult to judge.
- **Expressability:** *Unclear*. Without more information on the specifics, expressability is difficult to judge.

§ 6 Conclusion

Inadvertently reading an uninitialized nonclass variable on the program stack is a known source of difficult to diagnose bugs. Moreover, these defects lead to security holes that can be additionally problematic. One possible solution, proposed in [P2723R0], is to simply zero-initialize every automatic variable. Based on that initial suggestion, several other solutions have been proposed. We began this paper by enumerating some related issues involving solution scope concerning *dynamic memory*, *arrays*, *unions*, and *padding*. After identifying several useful diagnostic questions to elicit important distinguishing properties, we then proceeded to elucidate the various manifestations of this correctness and security problem with several small code examples. Finally, we identified seven different solution approaches and evaluated them against three separate criteria (*viability*, *backward compatibility*, and *expressability*), the results of which are summarized below.

Section	Proposed Solution	Viability	Backward Compatibility	Expressability
5.1	Always Zero-Initialize	Viable	Compatible	Worse
5.2	Zero-Initialize or Diagnose	Unclear	Correct-Code Compatible	Unchanged
5.3	Force-Initialize in Source	Viable	Incompatible	Better
5.4	Force-Initialize or Annotate	Viable	Incompatible	Better
5.5	Default Value, Still UB	Nonviable	Compatible	Unchanged
5.6	Default Value, Erroneous	Viable	Compatible	Unchanged
5.7	Value-Initialize Only	Unclear	Unclear	Unclear

Based on this analysis, we conclude that the baseline approach [section 5.1] of zero-initializing everything, similar to how static nonclass data is initialized, would be effective at plugging all such security holes but would add a meaningful definition to currently UB, which in turn would make diagnosing such inadvertent mistakes more difficult moving forward.

Combining zero initialization with compile time failure [section 5.2] has the serious drawback of some compilers accepting code which other compilers reject. Forced initialization, without [section 5.3] or with [section 5.4] annotation for intentional delayed initialization, imposes an enormous effort to modify existing code, even existing correct code. This change would encourage the cavalier use of scripts to explicitly default-initialize (or annotate) all previously uninitialized variables, thereby losing the intent of the original author. This strategy is likely to be met with resistance by many existing code bases. Requiring a defined meaning and behavior for UB [section 5.5] is nonviable, and recommending such behavior, though perfectly reasonable, simply cannot be enforced on an otherwise compliant implementation.

The *EB* approach [section 5.6] affords almost all the advantages of the others with few drawbacks. This strategy will, however, require some thought to introduce a new kind of behavior, EB, to the C++ abstract machine. Importantly and unlike many of the other proposals, defining *uninitialized memory reads* as EB provides no hindrance to longer-term solutions, such as the option of eliminating default initialization entirely [section 5.7].